Programmation en Java — Projet

L3 Informatique – 1er semestre – automne 2026 – Version 2, en date du 6 novembre 2026

Sujet : Réaliser le jeu Multitask 2 en Java

Le but de ce projet est de réaliser une version PC offline d'un petit jeu en ligne : **Multitask 2** (IcyLime, 2011). Il se joue au clavier uniquement : plusieurs mini-jeux se déroulent en même temps, chacun avec ses propres touches, et la partie s'arrête dès qu'un seul d'entre eux est perdu. Plus on tient longtemps, plus le score est élevé.
Bosphore, 13 ans et demi, aimerait jouer à Multitask 2 en dehors de son navigateur ; comme il n'est pas pressé, il est prêt à attendre la fin du semestre. Aidez-le !

Le jeu

Avant d'écrire la moindre ligne de code, **jouez au jeu original**, disponible en suivant lien ci-dessus : c'est lui qui définit les règles que vous devez reproduire. Si le jeu original et le présent sujet se contredisent, c'est le sujet qui prévaut.

Les six mini-jeux

Chaque mini-jeu se joue avec ses propres touches.

Mini-jeu	Touches	Principe
`Balance`	← →	Garder une bille sur une planche que l'on incline. *(fourni)*
`Avoid`	↑ ↓	Esquiver des triangles qui arrivent sur l'une des cinq lignes. *(fourni)*
`Touch`	W A S D	Déplacer un carré pour toucher des blocs avant la fin de leur compte à rebours.
`Fly`	Espace	Maintenir la touche pour monter, la relâcher pour descendre, et éviter les obstacles.
`Jump`	Maj	Sauter au bon moment par-dessus des trous dans le sol.
`Circle`	Q E	Faire tourner un cercle percé autour de son centre pour capturer des triangles qui arrivent de toutes les directions.

Remarque sur les claviers.

Les touches ci-dessus sont celles du jeu original, pensé pour un clavier QWERTY. Sur un clavier AZERTY, vous pouvez remplacer W A S D par Z Q S D et Q E par A E. Dans tous les cas, les touches de chaque mini-jeu doivent être définies *à un seul endroit* de votre code. On pourra prévoir un écran permettant à l'utilisateur d'indiquer s'il dispose d'un clavier AZERTY ou QWERTY.

Les trois modes de jeu

- ▷ **Standard** : la partie commence avec un seul mini-jeu, `Balance` ; les autres arrivent ensuite un par un, dans un ordre fixe, jusqu'à ce que les six soient en jeu.
- ▷ **Challenge** : trois mini-jeux, tirés au hasard et distincts, sont en jeu dès le début de la partie.
- ▷ **Switch** : les six mini-jeux sont en jeu, mais le joueur n'en contrôle qu'un seul à la fois ; le contrôle passe régulièrement d'un mini-jeu à un autre.

Les modes sont débloqués progressivement : le mode Challenge devient disponible dès que l'on a obtenu un score d'au moins 50 en mode Standard, et le mode Switch dès que l'on a obtenu au moins 50 en mode Challenge.

Score et difficulté

- ▷ Dans ce projet, le **score** d'une partie est le nombre de secondes écoulées avant la défaite.
- ▷ La **difficulté** augmente avec le temps ; chaque mini-jeu doit en tenir compte à sa manière (vitesses, fréquences d'apparition, écarts entre les obstacles, ...), comme dans le jeu original.

Le code fourni

On vous fournit un petit prototype qui utilise la bibliothèque graphique `zen` et suit le modèle de développement MVC (Modèle-Vue-Contrôleur) :

- ▷ Le *modèle* est formé de la classe `Model`, qui contient une liste de `Slot` (un mini-jeu et l'instant de son activation), et de l'interface `Game`, implémentée par chaque mini-jeu (`Balance` et `Avoid`). La méthode `update` d'un mini-jeu renvoie `true` tant que le mini-jeu n'est pas perdu.
- ▷ La classe `Input` garde en mémoire les touches actuellement enfoncées, et celles qui viennent d'être pressées pendant l'image courante.
- ▷ Le *contrôleur* (classe `Controller`) implémente la boucle de jeu (environ 60 images par seconde) : il lit les événements, met à jour le modèle, puis demande à la vue de se redessiner.
- ▷ La *vue* est l'interface `View`, implémentée par la classe `ZenGameView`. Elle est accompagnée d'une interface `GameView`, qui permet d'implémenter les vues propres à chaque mini-jeu, et d'une classe `Viewers`, qui associe à chaque mini-jeu le `GameView` à utiliser.

Ce prototype est volontairement sommaire :

- ▷ il ne contient que deux mini-jeux, simplifiés et en partie différents de ceux de l'original ; il conviendra de les modifier pour les rendre fidèles au jeu original ;
- ▷ le programme se ferme dès que l'on perd : il n'y a ni menu, ni écran de fin, ni possibilité de rejouer ;
- ▷ le score n'est pas géré, et il n'existe qu'un seul mode de jeu.

Quelques particularités de Zen à connaître :

- ▷ la fenêtre occupe tout l'écran et n'a pas de bouton de fermeture : c'est à votre programme de la fermer ;
- ▷ la lecture des événements et le dessin doivent se faire depuis le thread principal : n'utilisez pas d'autre thread pour cela ;
- ▷ les touches reconnues sont les flèches, les lettres A à Z, Espace, Maj, Ctrl, Alt et Échap ; il n'y a **ni touche Entrée, ni chiffres** ;
- ▷ les clics de souris sont également disponibles (`PointerEvent`).

Conseils

- ▷ Lisez l'ensemble de l'énoncé avant même de commencer quoi que ce soit !
- ▷ Le but de ce projet est de vous entraîner à programmer "objet" : exploitez les notions vues en cours.
- ▷ Prenez le temps de réfléchir à la conception avant de coder. Demandez-vous en particulier ce qu'il faudrait modifier dans votre code pour ajouter un septième mini-jeu, ou un quatrième mode : moins il y a de choses à modifier, mieux c'est.
- ▷ Le sujet est évolutif : respectez bien les phases de réalisation, mais gardez à l'esprit ce que vous devrez faire dans les phases suivantes lorsque vous faites des choix d'implantation !
- ▷ Ce n'est pas la qualité de l'interface graphique qui compte, mais la qualité de votre conception et de votre programmation. Des formes géométriques simples suffisent.

Le programme à réaliser

Vous allez réaliser votre jeu en trois phases. Chaque phase devra être terminée et fonctionnelle avant de passer à la phase suivante.

Phase 1 : Fidélité et cycle de jeu

- ▷ Adaptez les mini-jeux `Balance` et `Avoid` pour qu'ils se comportent comme dans le jeu original, y compris lorsque la difficulté augmente.
- ▷ Perdre une partie ne doit plus fermer le programme : un écran de fin affiche le score, et le joueur peut rejouer ou revenir au menu.
- ▷ Ajoutez un menu principal (jouer, quitter), utilisable au clavier ou à la souris.
- ▷ Faites en sorte qu'ajouter un nouveau mini-jeu se fasse en *ajoutant* des classes, sans modifier le moteur ni le code des autres mini-jeux (à l'exception de la ligne qui déclare le nouveau mini-jeu).

Phase 2 : Les six mini-jeux et le mode Standard

- ▷ Programmez les mini-jeux `Touch`, `Fly`, `Jump` et `Circle`. Comme les deux premiers, ils doivent être fidèles à l'original et réagir à la difficulté.
- ▷ Mettez en place le mode Standard complet, avec l'arrivée progressive des six mini-jeux.
- ▷ Ajoutez un écran d'aide qui rappelle les touches de chaque mini-jeu.

Phase 3 : Scores et autres modes

Réalisez les fonctionnalités suivantes, par ordre de priorité décroissante :

- ▷ **High scores** : pour chaque mode, le programme conserve les cinq meilleurs scores, chacun associé à un nom de trois lettres que le joueur saisit à la fin de la partie. Les scores doivent être enregistrés dans un fichier, de sorte qu'on les retrouve en relançant le programme, et être consultables depuis le menu. Un fichier absent ou invalide ne doit pas faire planter le programme.
- ▷ **Mode Challenge**, avec son mécanisme de déblocage. L'état de déblocage des modes doit être retrouvé au lancement du programme.
- ▷ **Mode Switch**, avec son mécanisme de déblocage.
- ▷ *Facultatif* : touches configurables, pause, ...

Évaluation et calendrier

- ▷ Ce projet est à faire en **binôme** (c'est-à-dire exactement **deux** personnes) d'un **même groupe de TP**. Vous indiquerez sur e-learning le nom de votre partenaire avant le jeudi **12 novembre** à **22:00**.
- ▷ **Ce projet n'est pas noté en tant que tel.** L'évaluation aura lieu le mardi **12 janvier** à **10:00**, en **conditions d'examen** et en **temps limité** : **2 heures**. Elle consistera à réaliser certaines des tâches effectuées pendant le projet (par exemple implanter correctement la gestion de la difficulté dans le mini-jeu `Balance`, ou stocker et afficher les high scores) ainsi qu'à ajouter un **autre mini-jeu**, issu de l'imagination des enseignants.

  Conséquence : même si le projet se fait à deux, **chacun** de vous doit être capable de refaire seul, et rapidement, chacune des tâches du projet. Répartir le travail sans comprendre et maîtriser ce que fait l'autre serait un mauvais calcul.
- ▷ Une **version bêta** doit être déposée sur e-learning au plus tard le dimanche **29 novembre** à **22:00**. Il est attendu, à cette étape du projet, que la phase 1 soit terminée et que la phase 2 soit sérieusement entamée. Plus vous aurez avancé dans le projet, plus on pourra vous faire de retours pertinents.

- ▷ Une **soutenance bêta**, **facultative et non évaluée**, sera organisée les lundi **30 novembre**, mardi **1er décembre** et mercredi **2 décembre**. Vous ferez une démonstration sur une machine des salles de TP et serez interrogés sur le projet. Le but de cette soutenance est de vous encourager si vous êtes sur la bonne voie, et de vous remettre dans le droit chemin sinon : profitez-en.

Rendu de la version bêta

Votre rendu devra consister en une archive au format `zip` : tout `rar`, `tar.gz`, `7z` ou autre ne sera pas ouvert. Cette archive contiendra :

- ▷ un répertoire `src` contenant les sources du projet ;
- ▷ un répertoire `lib` contenant les librairies dont dépend l'application ;
- ▷ un `jar` exécutable `Multitask2.jar`, qui fonctionne avec la commande `java -jar Multitask2.jar` lancée depuis le répertoire extrait de l'archive.

Cette archive aura pour nom `Nom1_Nom2_Multitask2.zip`, où les noms sont ceux des membres du binôme par ordre alphabétique. L'extraction de cette archive devra créer un répertoire intitulé `Nom1_Nom2_Multitask2` et contenant tous les éléments demandés ci-dessus.
Votre code doit **compiler**, et le jeu doit se lancer avec la commande ci-dessus.

Règles de programmation

Le projet n'est pas noté, mais ces règles sont celles que nous attendons de vous ; prenez dès maintenant l'habitude de les respecter.

- ▷ Votre code doit être propre, lisible et **modulable** : ajouter un mini-jeu, un mode ou une touche doit être aussi simple que possible.
- ▷ Aucune méthode ne doit comporter plus de 20 lignes.
- ▷ Pas de duplication de code, pas de code inutile, pas de variable globale, et respect des principes de programmation objet.
- ▷ Aucun champ ne doit avoir une visibilité autre que `private`, et toute méthode de visibilité `public` doit commencer par vérifier que ses arguments sont raisonnables. Par exemple, si une méthode lance une `NullPointerException`, celle-ci doit être due à un appel à `Objects.requireNonNull` qui aura détecté que l'argument proposé était nul.
- ▷ Les entrées/sorties doivent être gérées conformément au cours ; par exemple, n'utilisez surtout pas `java.io.File`.
- ▷ Vous ne devez utiliser aucune librairie externe autre que `zen`.
- ▷ Vous ne devez pas copier-coller de code trouvé sur Internet : de toute façon, à l'examen, vous devrez savoir tout écrire vous-mêmes.

Références

1. Le jeu **Multitask 2** ;
2. Les **entrées/sorties sur fichier** ;
3. La bibliothèque graphique **Zen**, ses **sources** et sa **documentation** ;
4. Un prototype **MemoryGame** où a été codé un jeu très simple utilisant `zen`.
5. Le code source d'un prototype **Multitask2** ;
6. Une **vidéo** indiquant comment intégrer `zen` à un projet avec Eclipse.